

I Don’t Code, I Delegate

Turning Claude Code into an agent team: workers, judges and one tired tech lead

Martin Janitor · Tech Lead, Visma Labs Košice

Sudo Tech Night · Košice · 12 Nov 2026

ABOUT ME

Hi, I'm Martin

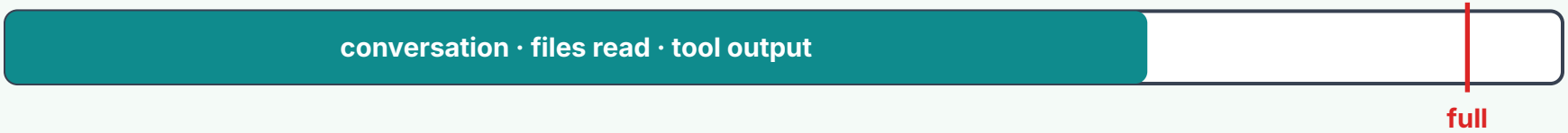
- Tech Lead, Visma Labs Košice
- Cloud web systems: Azure + .NET
- MTB and running

VISMA

A coding agent reads, runs and edits



CONTEXT WINDOW = ITS WORKING MEMORY



Compaction = when it's full, older turns are summarised. Detail, including your rules, can get lost.

Claude Code runs in your terminal or the desktop app.

VISMA

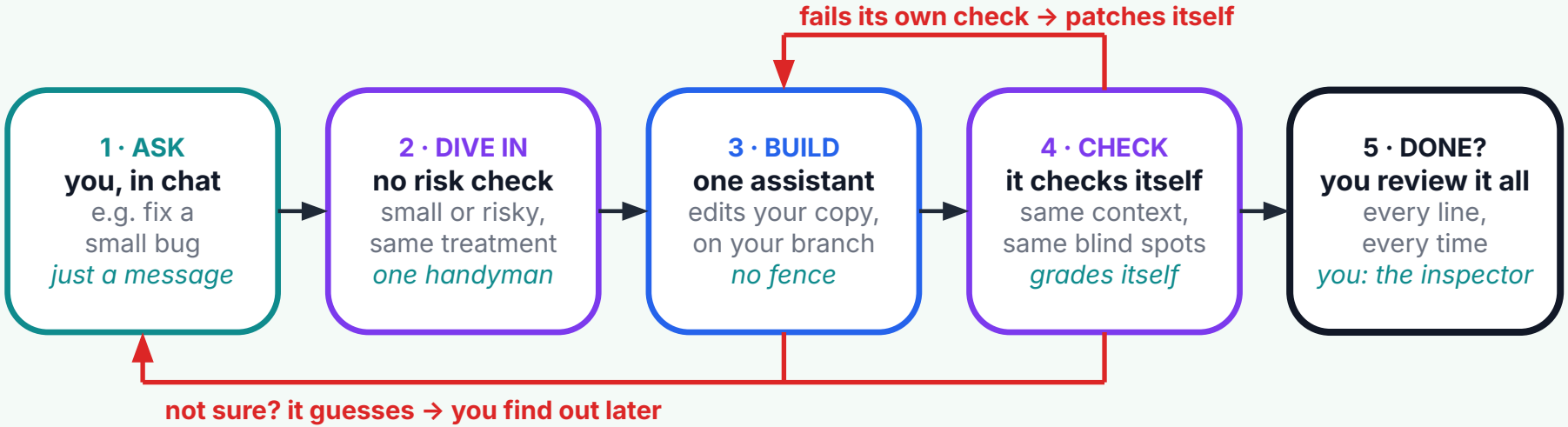
One assistant still hits walls

- Its memory fills up, then compacts
- It grades its own homework
- It guesses instead of asking
- It redoes what tools already do

LIFE OF ONE TASK · NO ORCHESTRATION

VISMA

One job, one assistant, one chat



Where it hurts
one memory = chat, files and tool output share one context; when it fills, your rules can get lost
one pair of eyes = the assistant that wrote the code also checks it, so you review everything

Coming up: the same five steps, with a team instead of one assistant.

WHAT ORCHESTRATION MEANS

Orchestration: a foreman, not a bricklayer

THEN

One assistant, I type too

NOW

A team, I mostly approve

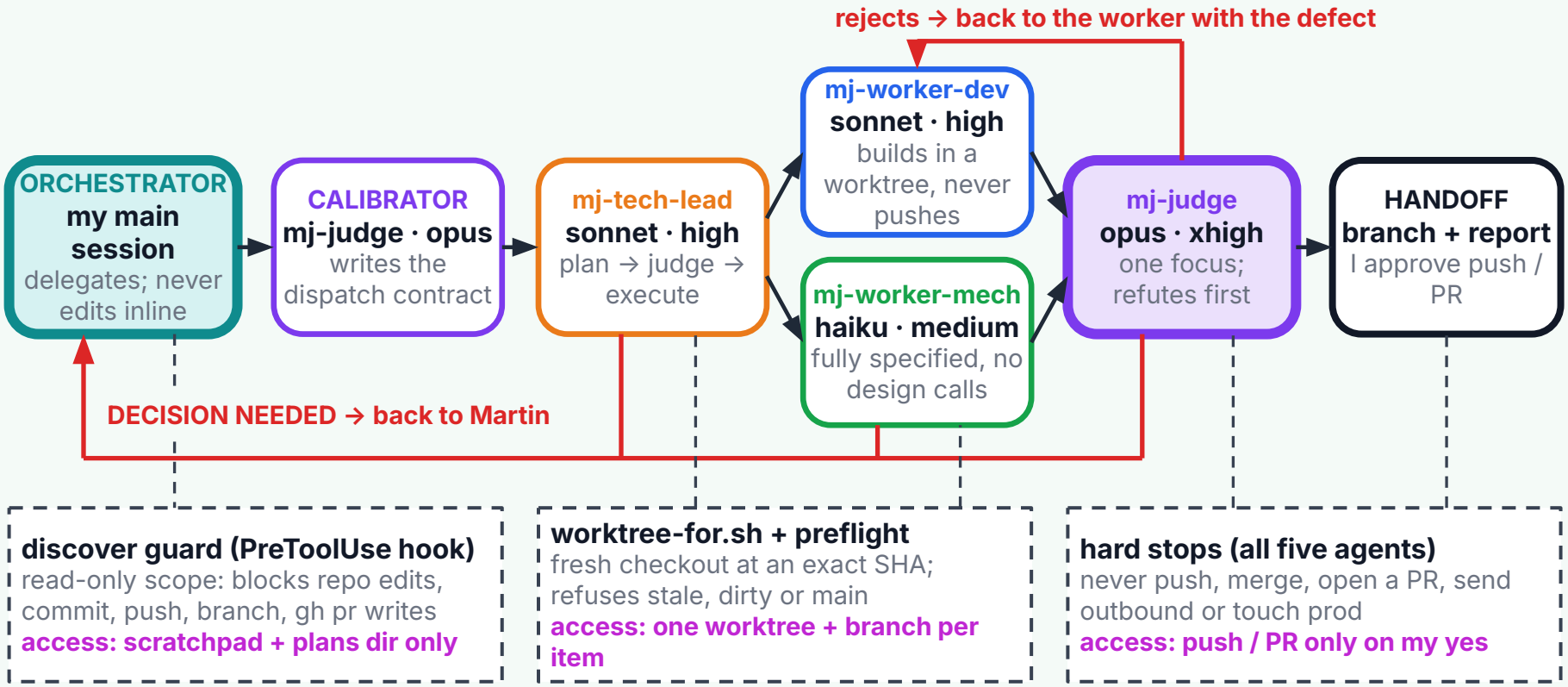
ORCHESTRATION, IN ONE LINE

One coordinator splits the work across specialised agents.

Like a site foreman running the crews.

VISMA

The team: workers, judges and one tired tech lead



≤5 agents at once · tech-lead default 3 concurrent / 6 total ·
judges never see the worker's reasoning · hooks fail open · 5th agent: mj-researcher (opus · high), outside-repo research

TOOL SELECTION

VISMA

Skill first, agent second, me last

`mj-orchestrate` · on top: routes, sizes, reports

1 · INSTALLED SKILL?

→ **use it**

`/deliver-work` — an orchestrator itself
our team's delivery plugin
8 phases · 8 gates · 11 agents · one lead
it owns the plan → I relay its gates, no 2nd plan
starts only when I type it · one item at a time

`/code-review` · `/security-review`
packaged judges: they count as the inspection, no extra mj-judge

2 · NO SKILL?

→ **mj-* agents**

`mj-tech-lead` →
`mj-worker-*` →
`mj-judge`

the fallback when a skill is missing

parallel items always go here

3 · NOTHING FITS?

→ **inline**

the orchestrator does it itself

and says so in the report

Missing deploy or release skill → stop and ask, never improvise.

Whatever the route, the result comes back into mj-orchestrate's report.

VISMA

Level: how often the inspector comes

STRICT

every brick inspected

plan approved before any code

2 judge checkpoints, 2+ focuses

every item, even small ones

for: irreversible work, releases

STANDARD · default

inspect by risk

S-tier decides (next slide)

clear small jobs take the fast path

at most 1 plan + 1 judge round

for: normal delivery

LIGHT

trusted crew, small jobs

no contract turn, no judges

plan only if asked

refuses S3 (irreversible) work

for: a batch of small, known items

Never changes with the level

hard stops · DECISION NEEDED · skill routing · push / merge / PR only on my yes

Switch any time: /mj-orchestrate strict|standard|light · Scope (build / discover) is separate: slide 11.

ORCHESTRATOR MODES · STAKES

VISMA

S-tiers: how bad if it's wrong?

S0 · MECHANICAL

fully specified, reversible

e.g. rename a config key

`mj-worker-mech`

no judge

S1 · ROUTINE

local, reversible

e.g. fix a small bug

`mj-worker-dev`

1 judge focus

S2 · TEAM-FACING

ships to the team: PR, release, tester handoff

plan judged first, premises tested

2–3 judge focuses

S3 · IRREVERSIBLE

irreversible or architecture

full panel, 3–4 focuses, judge at max effort

→ DECISION NEEDED to me

Fast path
one clear S0/S1 item, local and reversible, nothing to ask → straight to work, tier said in one line

The tier is set before any agent runs: that's the dispatch contract.

VISMA

Scope: what the crew may touch

BUILD · default

the crew builds

workers edit code in worktrees

one branch + commits per item

push / PR only on my yes

for: getting work done

DISCOVER

survey only, nothing built

reads repos, tickets, logs, web

no edits, commits or branches

returns findings + a draft plan

for: investigation and analysis

Discover is enforced three ways

the charter text · a "mode: discover (read-only)" line in every dispatch · the guard hook

Switch any time: /mj-orchestrate build|discover · Level is separate: discover works at any level. The demo runs in build.

DECISION PROTOCOL

Best feature: DECISION NEEDED

```
DECISION NEEDED
Question
Options (with evidence)
Recommendation
Blocked · Parked-and-done
```

- Never assume: prod, contracts, cross-repo
- The turn ends; "postpone" is valid
- 3 rejected plans = wrong ask

WHY THE HOOKS EXIST

VISMA

What the hooks do while I'm prompting

Hook = a small script Claude Code runs by itself at a set moment.

1 · OPEN A SESSION

`SessionStart`
mode-reset.sh

sets this session's mode to off

why: no level left over from yesterday

2 · I TYPE A PROMPT

`UserPromptSubmit`
mode-inject.sh

adds one line the model reads:

```
Orchestrator mode is ON: standard · build. Follow the /mj-orchestrate charter…
```

why: survives compaction; silent when off

3 · CLAUDE WANTS TO EDIT OR RUN BASH

`PreToolUse`
discover-guard.sh

build: passes · discover: blocks writes

```
discover scope is read-only: stash change. Run /mj-orchestrate build to leave it.
```

why: a backstop if the rule is missed

4 · ALL THE TIME

`UI module`
band + panel

shows 🎛️ orch standard · 🔨 build; click to switch

why: the desktop app draws no status line

The mode file · per session: level | time | scope · reset writes; inject, guard, band, panel read; /mj-orchestrate updates

Every hook fails open: a broken hook never blocks a session.

VISMA

Switching it on

```
/mj-orchestrate standard build
```

level scope

New session = off · plain Claude Code until you ask; a hook resets it at every start

`/mj-orchestrate` · opens a picker: level + scope

`/mj-orchestrate standard build <task>` · switches on and starts the task

Band or panel · shows the mode, click to switch · /mj-orchestrate off to leave

Level and scope: slides 9 and 11.

LIVE

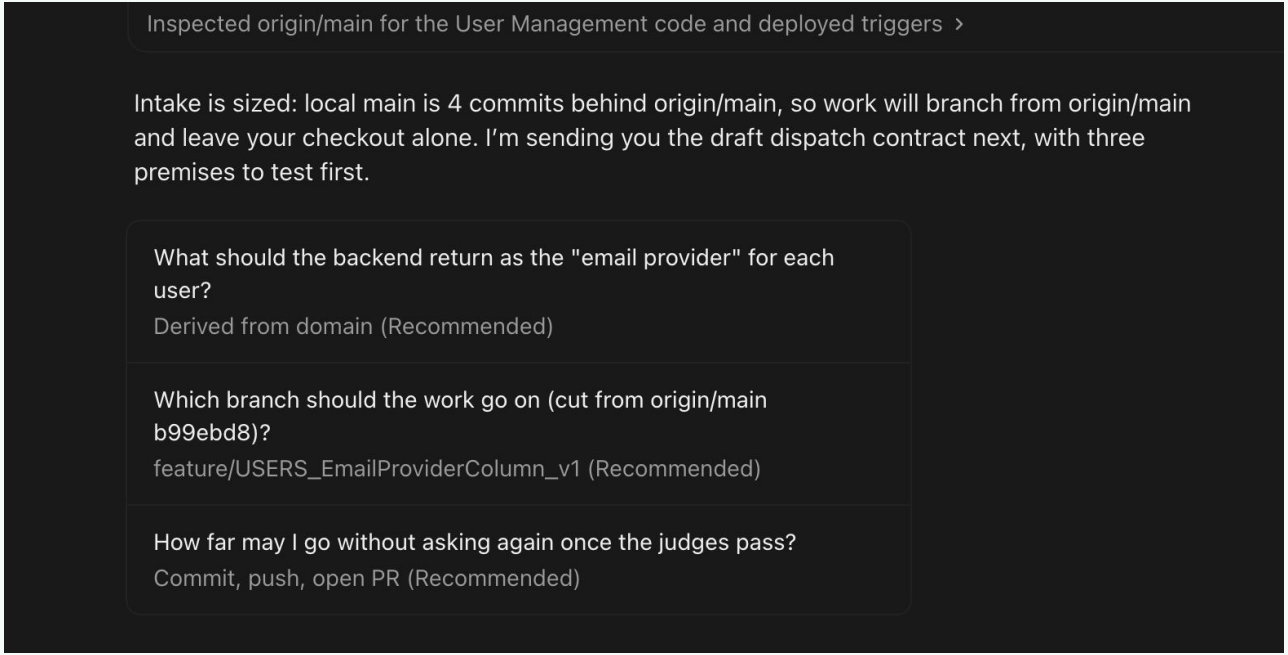
Live demo

One small feature · four agents · ~4 minutes

1	2	3	4
Size it	Two workers	Judge	Open the PR

FROM THE FULL RUN · DECISIONS FIRST

It asks before it builds



Three questions, each with a recommended answer. Nothing runs until I rule.

BACKUP · SKIP UNLESS DEMO FAILS

What you would have seen

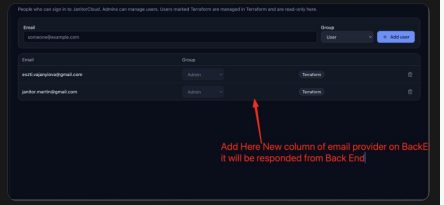
/mj-orchestrate:mj-orchestrate standard build

Persist orchestrator mode and read the routing overlay ›

Orchestrator mode ON — standard · build. Mode file written, local routing overlay loaded (this is a personal repo, so single items route to `mj-tech-lead` → `mj-worker-dev` → `mj-judge`, not `/deliver-work`).

What this changes: repo work is delegated to worktree subagents under a sized dispatch contract, S0/S1 items take the fast-path, S2+ get a plan and judge round, and nothing is pushed, merged or PR'd without your yes in that turn.

No open delegations, no unruled decisions. What should I orchestrate?



Hi Let's build an demo of how the orcherstoer works. From image you can see a very simple implementaion to implement a task. The goal is to show how the task are ochestrated. So you can put so ↓ nfo. And in the end automatically deploy it to the enviroenmnt and keep changes ..n branch.

Inspected origin/main for the User Management code and deployed triggers ›

Intake is sized: local main is 4 commits behind origin/main, so work will branch from origin/main and leave your checkout alone. I'm sending you the draft dispatch contract next, with three premises to test first.

What should the backend return as the "email provider" for each user?
Derived from domain (Recommended)

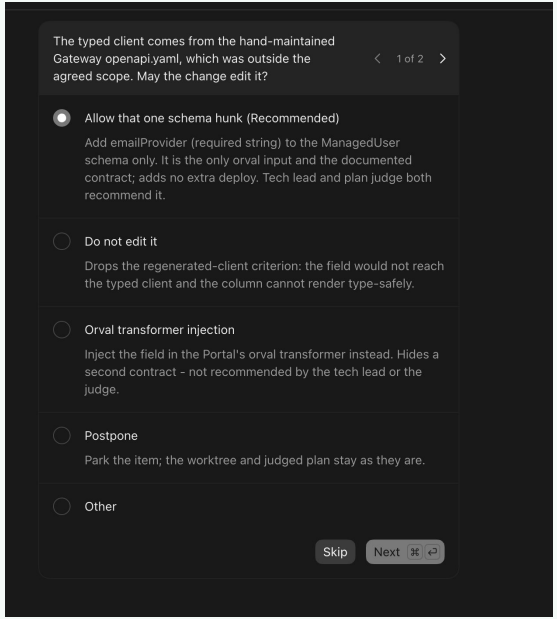
Which branch should the work go on (cut from origin/main b99ebd8)?
feature/USERS_EmailProviderColumn_v1 (Recommended)

How far may I go without asking again once the judges pass?
Commit, push, open PR (Recommended)

Recorded at rehearsal · same repo, same commands.

BACKUP · SKIP UNLESS DEMO FAILS

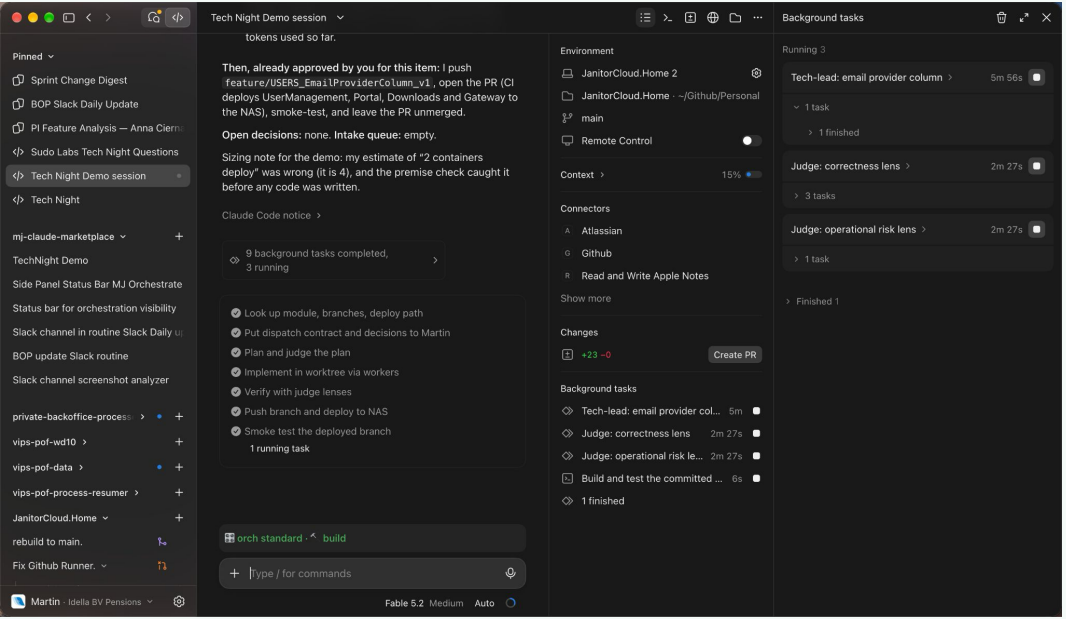
What you would have seen



Recorded at rehearsal · same repo, same commands.

BACKUP · SKIP UNLESS DEMO FAILS

What you would have seen



Dispatched. This is the orchestration in motion:

Stage	Who	State
Intake + sizing (S2)	orchestrator	done
Decisions flushed before work	you	ruled: derived from domain · `feature/USERS_EmailProviderColumn_v1` · commit + push + PR, no merge
Worktree from `origin/main` b99ebd8	script	done
Premises → plan → plan judge (spec lens)	`mj-tech-lead` + `mj-judge`	**running**
Implement against approved plan	`mj-worker-dev`	queued
Handoff judges: correctness + operational risk	2× `mj-judge`	queued
Push, open PR (CI deploys branch to NAS), smoke test	orchestrator	queued, pre-approved

Open delegations

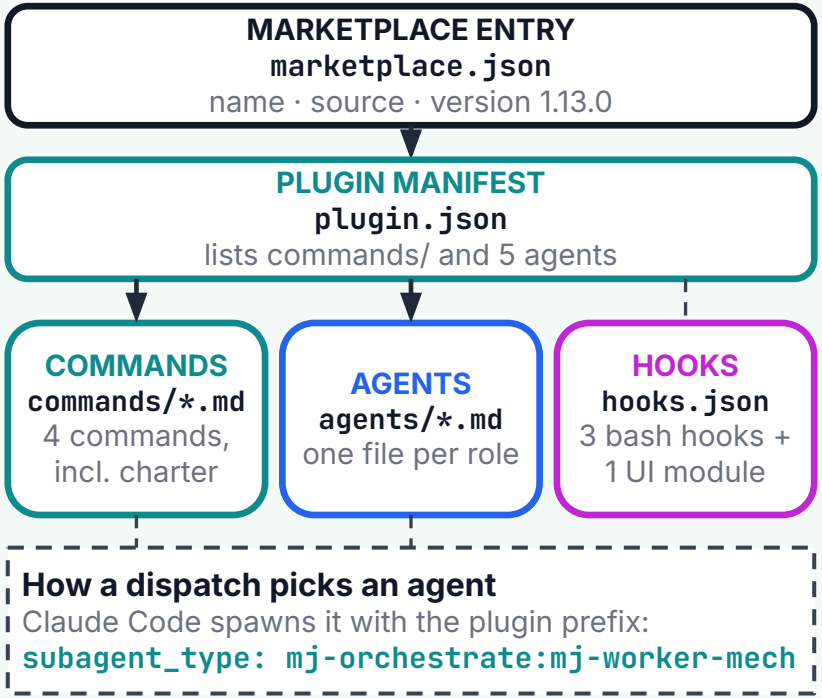
Recorded at rehearsal · same repo, same commands.

VISMA

An agent is a Markdown file

```
# agents/mj-worker-mech.md (trimmed)
---
name: mj-worker-mech
description: Mechanical delegation worker … No
design judgment.
model: haiku
effort: medium
color: green
disallowedTools: …
---
You are a mechanical worker … Do exactly what the
task says; …
HARD STOPS — never violated …: never push, never
open a PR, never merge …
DECISION PROTOCOL — never assume on: …
```

Same six keys in all five agents; the body is plain English. A removed tool fails as unavailable, not as a rule to remember.



No SDK, no framework code: Markdown and bash. Any git repo, any language.

Needs Claude Code, git, bash and jq (macOS/Linux). Band and panel are .tsx on Claude Code's own module API, running inside Claude Code, not in your repo.

SESSIONS END, WORK DOESN'T

VISMA

Keep, recall, review: memory that survives

Each runs only when you ask: nothing auto-archives.

/mj-keep

Saves this session to an archive

masked transcript + digest

filed by profile, then workspace

run again to refresh

WHY
Sessions end; context is lost. Keep, then start fresh cheaply.

/mj-recall

Answers from kept sessions

every claim cited and graded

pick one to continue

said then ≠ still true

WHY
Long multi-agent days must be resumable.

/mj-session-review

The end-of-day site meeting

done · decided · failed · open

proposes memory edits as a diff

writes only on my yes

WHY
A wrong memory costs more than a missing one.

Secrets become [REDACTED] in both files · files 0600 · keep the folders private · masking is a safety net, not a guarantee

TRADE-OFFS

The price, and what pays for it

- More tokens → sized before it runs
- More ceremony → fast path for small jobs
- Briefs take effort → that's the contract
- Rules can be forgotten → hooks back them up
- It needs upkeep → every fix is a version

TAKEAWAYS

Steal these five

1. Delegate by default
2. Skill first, agent second, me last
3. Make "I'm not sure" an output
4. Instructions are control, hooks are seatbelts
5. Review before you remember

VISMA

Thanks · Questions?

Martin Janitor

Tech Lead, Visma Labs Košice

THE PLUGIN, EXPORTED AS A MARKETPLACE

github.com/Domain-Factory-SK/tech-night-orchestrator

An example you can take inspiration from.